

Exercise 1

Let V be a subspace of dimension d of a Hilbert space H . Show that the projection of $x \in \mathbb{R}^p$ onto V can be written as

$$\pi_V(x) = \sum_{i=1}^d \langle x, v_i \rangle v_i$$

where v_i form an orthonormal basis, B for V .

The following properties hold for V :

1. If $x \in V$, $\pi_V(x) = x$
2. For any $v \in V$, $x \in \mathbb{R}^p$, $\langle x - \pi_V(x), v \rangle = 0$

Proof: Let v_i be a orthonormal basis, B for V . Since $\pi_V(x) \in V$, $\pi_V(x)$ can be written as

$$\pi_V(x) = \sum_i \alpha_i v_i$$

for $\alpha_i \in \mathbb{R}$. Then by property 2 and for any element $v_j \in B$ we have

$$\begin{aligned} \langle x - \sum_i \alpha_i v_i, v_j \rangle &= 0 \implies \\ \langle x, v_j \rangle &= \langle \sum_i \alpha_i v_i, v_j \rangle = \sum_i \alpha_i \langle v_i, v_j \rangle = \sum_i \alpha_i \delta_{ij} = \alpha_j \end{aligned}$$

$$\text{Thus } \alpha_j = \langle x, v_j \rangle \text{ and } \pi_V(x) = \sum_i \alpha_i v_i = \sum_i \langle x, v_i \rangle v_i$$

Exercise 3

$$R(x) = \frac{x M x'}{x^\top x}$$

Show that $R(x)$ is maximized when it equals the largest eigenvalue of M . This occurs when x is the associated eigenvector.

Let $M = P \Lambda P^\top$ be the spectral decomposition of M . The columns of P are the eigenvectors of M and form an orthonormal basis, B of dimension d . The dimension d is equal to the number of eigenvectors with non-zero eigenvalues. We may complete B by augmenting it by $(p-d)$ additional vectors that are orthonormal to each other and to B . Likewise, we also extend Λ as

$$\Gamma = \begin{bmatrix} \lambda_1 & & & & 0 \\ & \ddots & & & \\ & & \lambda_d & & \\ & & & 0 & \\ 0 & & & & \ddots & \\ & & & & & 0 \end{bmatrix}$$

Then M can be written

$$M = Q \Gamma Q^\top$$

with $QQ^\top = I$ and $Q^\top Q = I$. Defining $y = Q^\top x$, $R(x)$ can be written

$$R(x) = \frac{y^\top \Gamma y}{y^\top y} = \frac{\sum_{i=1}^p \lambda_i y_i^2}{\sum_{j=1}^p y_j^2} = \sum_{i=1}^p \lambda_i \left(\frac{y_i^2}{\sum_{j=1}^p y_j^2} \right)$$

If we set $w_i = \frac{y_i^2}{\sum_{j=1}^p y_j^2}$ then $\sum_i w_i = 1$ and $R(x)$ can be seen to be a

convex combination of the eigenvalues of M . This implies that $R(x)$ will reach a maximum when $R(x) = \lambda_{\max}$, and all the terms in the sum are zero except for this one. If this index is the j th then $w_{\max} = 1_j$. If we order the eigenvalues $\{v_1, \dots, v_d\}$ from largest to smallest, then $\max_{x \in \mathbb{R}^p} R(x) = \lambda_1$ and $\operatorname{argmax} R(x) = v_1$.

Exercise 6

The centered kernel is given by

$$\tilde{K}(x, y) = \left\langle K(\cdot, x) - \frac{1}{n} \sum_i K(\cdot, x_i), K(\cdot, y) - \frac{1}{n} \sum_i K(\cdot, x_i) \right\rangle = \left\langle \frac{1}{n} \sum_i (K(\cdot, x) - K(\cdot, x_i)), \frac{1}{n} \sum_i (K(\cdot, y) - K(\cdot, x_i)) \right\rangle$$

Show that is symmetric and p.d.

$\tilde{K}(x, y) = \tilde{K}(y, x)$ follows because the inner product is symmetric.

Positive definiteness follows from the following:

$$\begin{aligned} \sum_{kl} \alpha_k \alpha_l \tilde{K}(x_k, x_l) &= \left\langle \frac{1}{n} \sum_i \left(\sum_k \alpha_k K(\cdot, x_k) - K(\cdot, x_i) \right), \frac{1}{n} \sum_i \left(\sum_l \alpha_l K(\cdot, x_l) - K(\cdot, x_i) \right) \right\rangle = \\ &= \left\| \frac{1}{n} \sum_i \sum_k \alpha_k (K(\cdot, x_k) - K(\cdot, x_i)) \right\|^2 = \left\| \sum_k \alpha_k \left(K(\cdot, x_k) - \frac{1}{n} \sum_i K(\cdot, x_i) \right) \right\|^2 \geq 0 \end{aligned}$$

Exercise 7

Show that that the RKHS of $\tilde{K}$ is given by the following:

$$\tilde{H} = \left\{ f_g(\cdot) = g(\cdot) - \frac{1}{n} \sum_i g(x_i), g \in H \right\}$$

with $\langle f_{g_1}, f_{g_2} \rangle = \langle g_1, g_2 \rangle$

We see that $\tilde{K} = f_K \in \tilde{H}$ since H is a vector space which implies

$$K(\cdot, x) - \frac{1}{n} \sum_i K(\cdot, x_i) \in H$$

Also $\tilde{K}$ has the reproducing property since

$$\begin{aligned}\langle \tilde{K}(\cdot, x), f_g \rangle_{\tilde{H}} &= \langle f_{K(\cdot, x)}, f_g \rangle_{\tilde{H}} = \\ \langle K(\cdot, x), \frac{1}{n} \sum_i (g(\cdot) - g(x_i)) \rangle_H &= \frac{1}{n} \sum_i (g(x) - g(x_i)) = \\ g(x) - \frac{1}{n} \sum_i g(x_i) &= f_g(x)\end{aligned}$$

The second line follows from the reproducing property of H .

Exercise 8

Show that for any $f \in \tilde{H}$, $\sum_i^n f(x_i) = 0$

Proof: $f_g \in \tilde{H} \implies g \in H$. Therefore,

$$\begin{aligned}\frac{1}{n} \sum_i^n f(x_i) &= \frac{1}{n} \sum_i^n g(x_i) - \frac{1}{n^2} \sum_{ij} g(x_j) = \\ \frac{1}{n} \sum_i^n g(x_i) - \frac{n}{n^2} \sum_j g(x_j) &= 0\end{aligned}$$

Exercise 9

Defining $w_j = \Lambda^{\frac{1}{2}} P^\top \alpha_j$. Show that in PCA, the optimization problem can be written as

$$\text{maximize } \sum_{j=1}^d w_j^\top \Lambda w_j \text{ with } w_i^\top w_j = \delta_{ij}$$

Since $w_j^\top = (\Lambda^{\frac{1}{2}} P^\top \alpha_j)^\top = \alpha_j^\top P \Lambda^{\frac{1}{2}}$ and $PP^\top = I$ we have

$$K = P^\top \Lambda P \implies K^2 = (P^\top \Lambda P)(P^\top \Lambda P) = (P^\top \Lambda^{\frac{1}{2}}) \Lambda (\Lambda^{\frac{1}{2}} P) \implies$$

$$\sum_j \alpha_j K^2 \alpha_j = \sum_j w_j^\top \Lambda w_j$$

Also $\alpha_j^\top K \alpha_k = \alpha_j^\top P^\top \Lambda^{\frac{1}{2}} \Lambda^{\frac{1}{2}} P \alpha_k = \delta_{jk} \implies w_j^\top w = \delta_{jk}$ is the constraint in terms of w .

Thus as suggested above, the PCA optimization problem can be reframed in terms of the variable w .

Exercise 10

Use the procedure in exercise 3 to show that the solution to the kernelized PCA is w_1 .

Let $\Gamma = \text{diag}(\lambda_1, \dots, \lambda_d, 0, \dots, 0)$ where $\lambda_1 \leq \lambda_2 \leq \dots \leq 0$ In this case $Q = P$ and $M = I\Gamma I$ and $y = Qw = Iw = w \implies$

$$R(x) = \sum_{i=1}^p \lambda_i \frac{y_i^2}{\sum_{j=1}^p y_j^2} \equiv \sum_{i=1}^p \lambda_i z_i$$

Once again for $R(x)$ is a convex combination of the eigenvalues of M . This implies that $R(x)$ will reach a maximum when $R(x) = \lambda_{\max}$, so all the terms in the sum are zero except for, say the j th so $y_{\max} = I w_{\max} = \mathbb{1}_j$. The PCA algorithm aims maximize all $R(x)$ over a set of points $\{x_1 \dots x_d\}$ where each point $x_i \in \mathbb{R}^d$. This is equivalent to finding the points that maximize the sum

$$S = \text{maximize} \sum_{i=1}^d x_i^\top M x_i \text{ such that } \langle x_i, x_j \rangle = \delta_{ij}$$

From the discussion above, we know that the solution is set the normalized eigenvectors $\{v_1, v_2, \dots, v_d\}$ of K , ordered from largest to smallest by eigenvalue.

2 Kernel centering

Let $\mathcal{X} = \mathbb{R}^d$. Show that for $K(x, y) = x^\top y$ the centered kernel is given by

$$\tilde{K}(x, y) = (x - \bar{x})^\top (y - \bar{x})$$

with

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

$$\begin{aligned} \tilde{K}(x, y) &= \langle K(\cdot, x) - \frac{1}{n} \sum_i K(\cdot, x_i), K(\cdot, y) - \frac{1}{n} \sum_i K(\cdot, x_i) \rangle = \\ &\langle K(\cdot, x), K(\cdot, y) \rangle - \frac{1}{n} \sum_i \langle K(\cdot, x_i), K(\cdot, y) \rangle - \frac{1}{n} \sum_i \langle K(\cdot, x_i), K(\cdot, x) \rangle + \\ &\frac{1}{n^2} \sum_{ij} \langle K(\cdot, x_i), K(\cdot, x_j) \rangle = \\ &x^\top y + \frac{y}{n} \sum_i \bar{x}_i^\top - \frac{x}{n} \sum_i \bar{x}_i^\top + \left(\frac{1}{n} \sum_i \bar{x}_i^\top \right) \left(\frac{1}{n} \sum_j \bar{x}_j \right) = \\ &x^\top y - \bar{x}^\top y - \bar{x}^\top x + \bar{x}^\top \bar{x} = (x^\top - \bar{x}^\top)(y - \bar{x}) = (x - \bar{x})^\top (y - \bar{x}) \end{aligned}$$

3. Coding

STAT672_Homework1_code

January 25, 2025

```
[17]: import math
import random
import numpy as np
import matplotlib.pyplot as plt

#generate some random data
def generate_points_on_circle(radius, num_points, noise):
    xpoints=[]
    ypoints=[]
    for _ in range(num_points):
        # Generate a random angle in radians
        theta = random.uniform(0, 2 * math.pi)

        # Calculate x and y coordinates using polar coordinates
        x = (radius+np.random.normal(0,noise,1)) * math.cos(theta)
        y = (radius+np.random.normal(0,noise,1)) * math.sin(theta)

        xpoints.append(x)
        ypoints.append(y)

    return np.array(xpoints), np.array(ypoints)
#Parameters
categories=3
factor=25
radius=[1,4,6]
color=['green', 'blue', 'red']
noise=.3
#Construct X(:,0) and X(:,1)
Xp=np.empty((0,1))
Yp=np.empty((0,1))

for c in range(categories):
    num_points=20*radius[c]
    xpoints,ypoints = generate_points_on_circle(radius[c],
    ↪int(num_points),noise)
    Xp=np.vstack((Xp,xpoints))
    Yp=np.vstack((Yp,ypoints))
```

```

    #Scatter plot each color category
    plt.scatter(xpoints,ypoints,s=2,color=color[c])
plt.axis('equal') # To maintain aspect ratio
plt.title('Random Points on Circle')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.show()

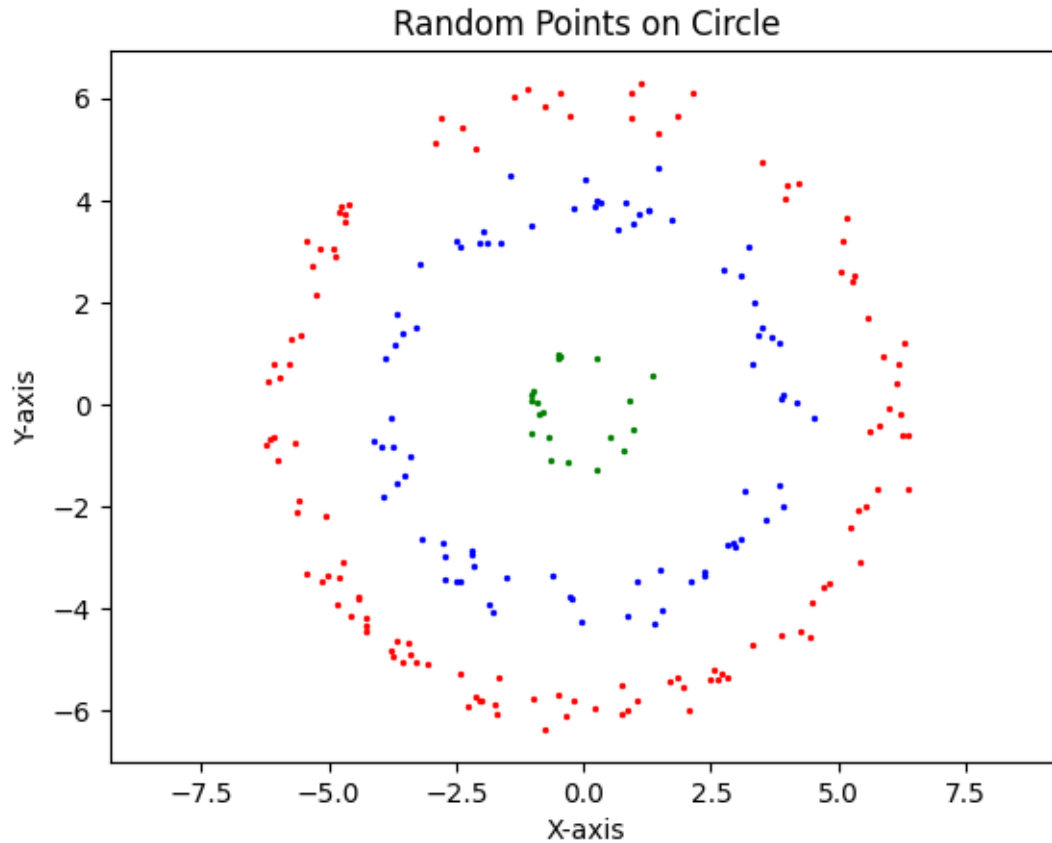
#Construct X Matrix from data
X=np.hstack((Xp,Yp))
n_samples = X.shape[0]
One_matrix=np.ones((n_samples,n_samples))
Id_matrix=np.identity((n_samples))
#Variety of Kernels for PCA
def polynomial_kernel(X, Y, degree=2, c=1):
    return (np.dot(X, Y) + c) ** degree

def rbf_kernel(X, Y, gamma=1):
    return np.exp(-gamma * np.linalg.norm(X-Y)**2)

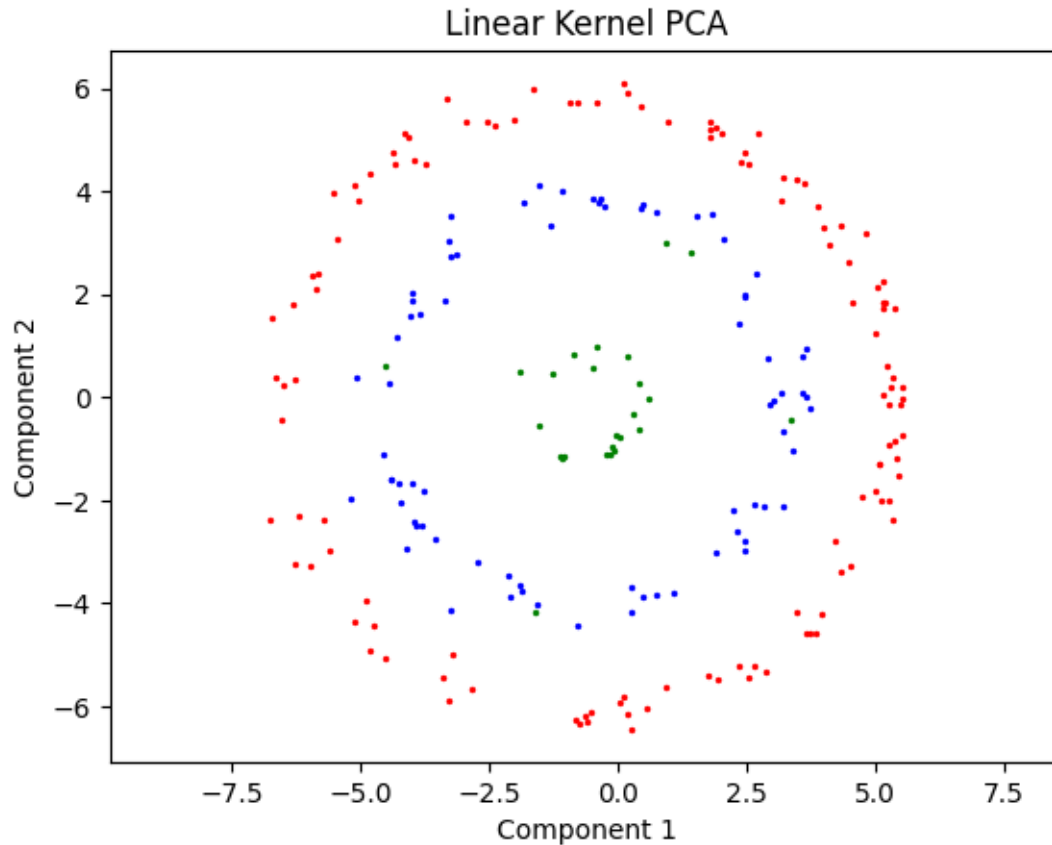
#Create Kernel Matrix
def create_kernel_matrix(X, kernel_func, **kwargs):
    n_samples = X.shape[0]
    One_matrix=np.ones((n_samples,n_samples))
    Id_matrix=np.identity((n_samples))
    K = np.zeros((n_samples, n_samples))
    for i in range(n_samples):
        for j in range(n_samples):
            K[i, j] = kernel_func(X[i], X[j], **kwargs)
    Kc=(Id_matrix-One_matrix/n_samples)@K@(Id_matrix-One_matrix/n_samples)
    return K,Kc

#Sort eigenvalues and eigenvectors from largest to smallest.
def eig_sorted(A):
    eigenvalues, eigenvectors = np.linalg.eig(A)
    idx = np.argsort(eigenvalues)[::-1] # Sort in descending order
    eigenvalues = eigenvalues[idx]
    eigenvectors = eigenvectors[:, idx]
    return eigenvalues, eigenvectors

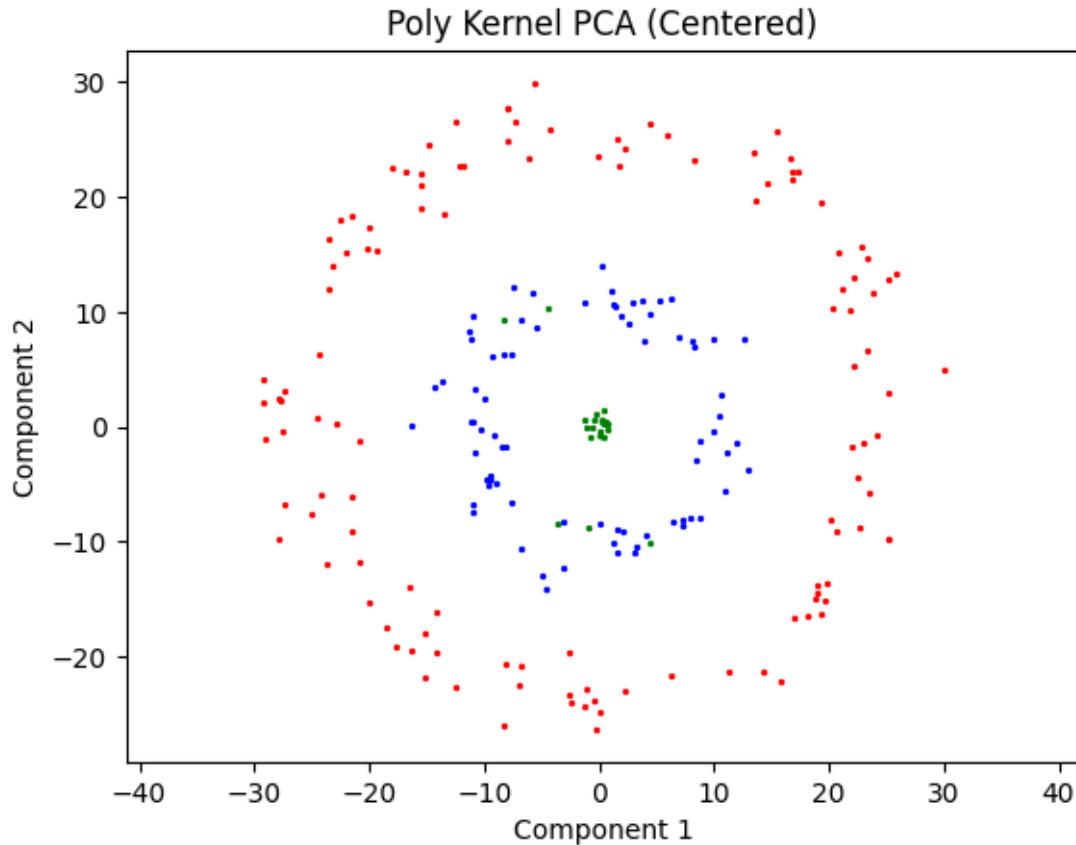
```



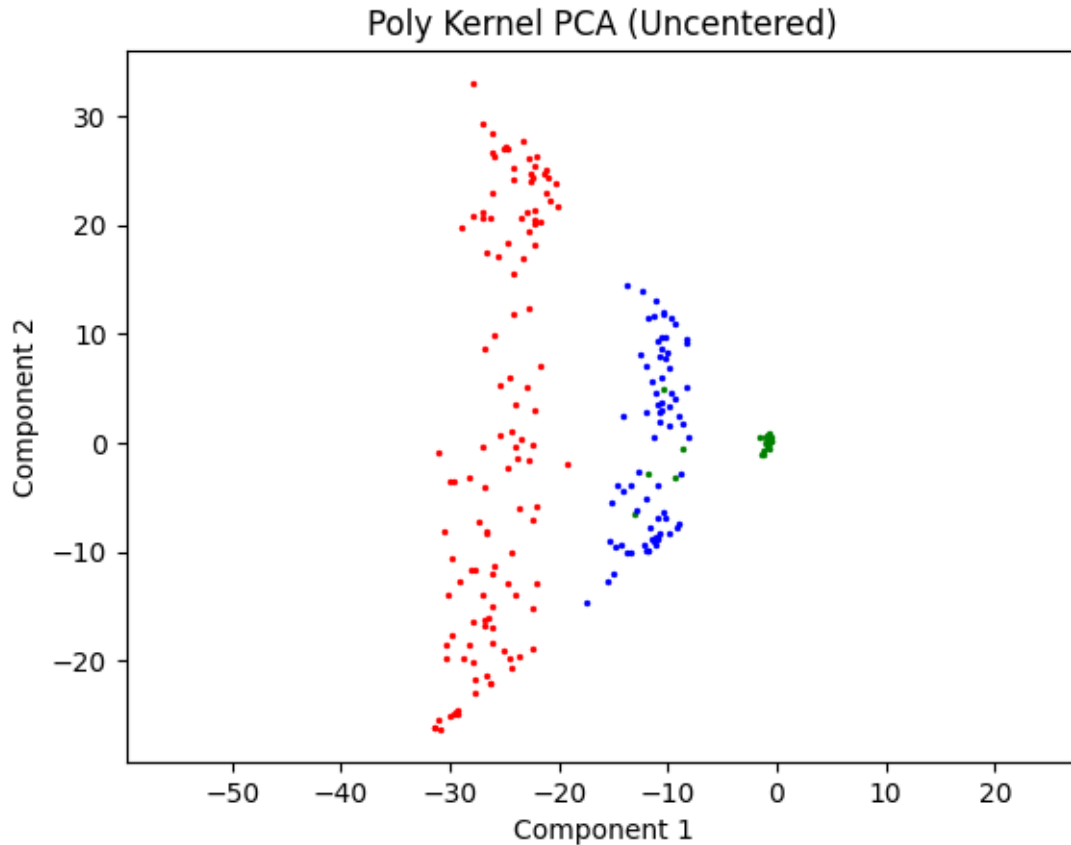
```
[26]: #Linear PCA class method
K_poly,Kc_poly = create_kernel_matrix(X, polynomial_kernel, degree=1, c=0)
eigenvalues, eigenvectors=eig_sorted(Kc_poly)
pvalues=np.real(eigenvalues[0:2])
pvectors=np.real(eigenvectors[:,0:2])
D=np.diag(1/np.sqrt(pvalues))
PCA=Kc_poly@pvectors@D
inter=[slice(0,int(radius[0]*factor)),slice(int(radius[0]*factor),radius[1]*\
        factor),slice(radius[1]*factor,PCA.shape[0])]
for c in range(categories):
    plt.scatter(PCA[inter[c],0],PCA[inter[c],1],s=2,color=color[c])
plt.title("Linear Kernel PCA")
plt.xlabel('Component 1')
plt.axis('equal')
plt.ylabel('Component 2')
plt.show()
```



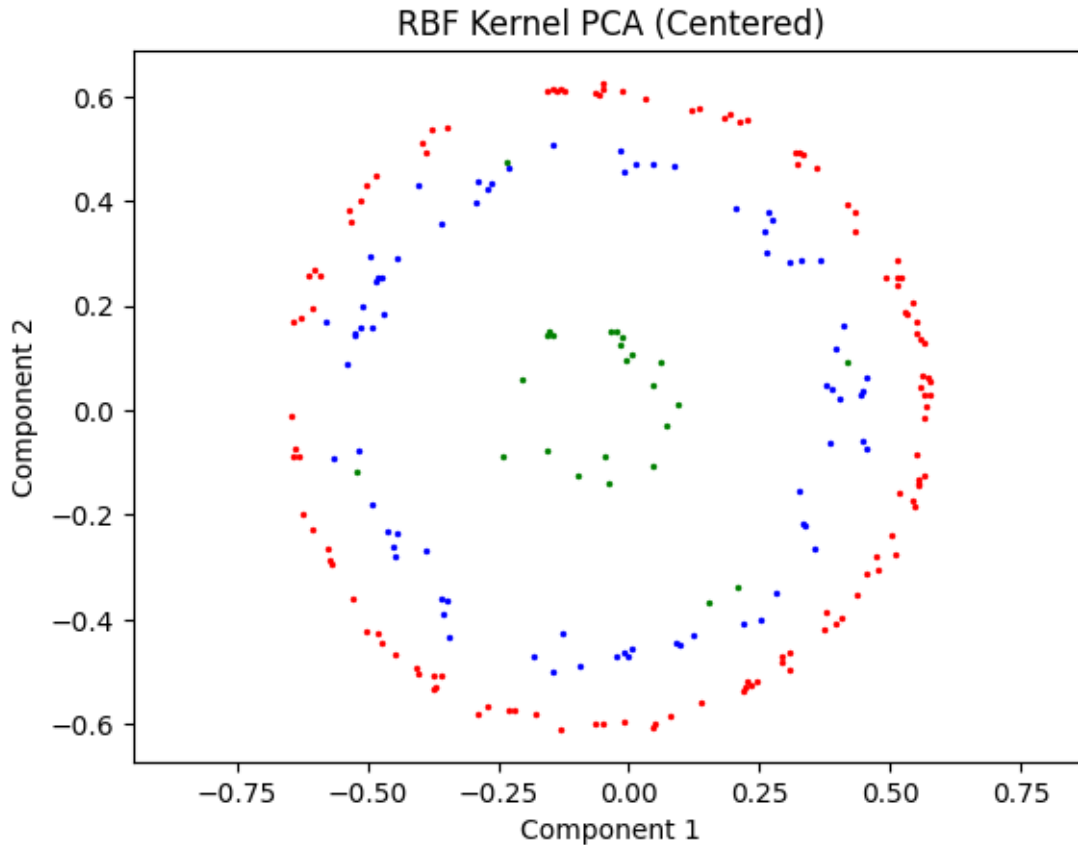
```
[27]: # Polynomial PCA class method
K_poly, Kc_poly = create_kernel_matrix(X, polynomial_kernel, degree=2, c=1)
eigenvalues, eigenvectors=eig_sorted(Kc_poly)
pvalues=np.real(eigenvalues[0:2])
pvectors=np.real(eigenvectors[:,0:2])
D=np.diag(1/np.sqrt(pvalues))
PCA=K_poly@pvectors@D
inter=[slice(0,int(radius[0]*factor)),slice(int(radius[0]*factor),radius[1]*\
        factor),slice(radius[1]*factor,PCA.shape[0])]
for c in range(categories):
    plt.scatter(PCA[inter[c],0],PCA[inter[c],1],s=2,color=color[c])
plt.title("Poly Kernel PCA (Centered)")
plt.xlabel('Component 1')
plt.ylabel('Component 2')
plt.axis('equal')
plt.show()
```

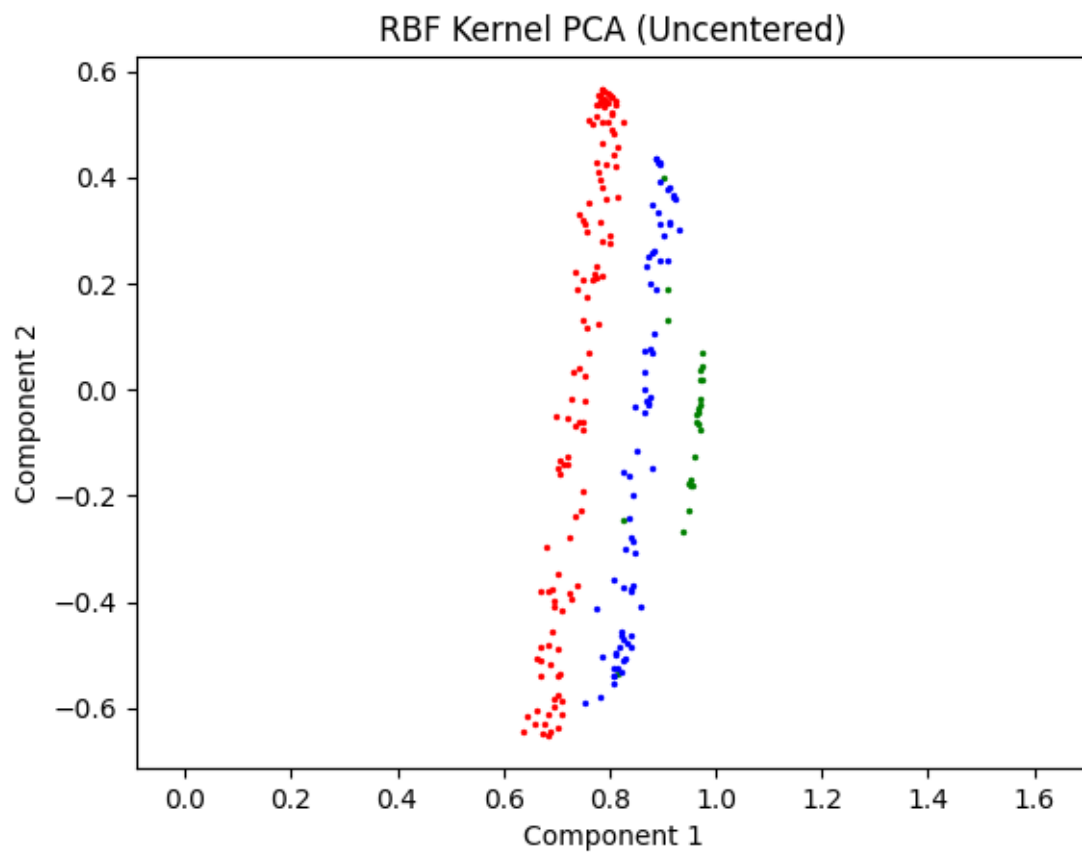
```
[28]: # Polynomial PCA class method (uncentered)
K_poly, Kc_poly = create_kernel_matrix(X, polynomial_kernel, degree=2, c=1)
eigenvalues, eigenvectors = eig_sorted(K_poly)
pvalues = np.real(eigenvalues[0:2])
pvectors = np.real(eigenvectors[:, 0:2])
D = np.diag(1/np.sqrt(pvalues))
PCA = K_poly @ pvectors @ D
inter = [slice(0, int(radius[0]*factor)), slice(int(radius[0]*factor), radius[1]*\
        factor), slice(radius[1]*factor, PCA.shape[0])]
for c in range(categories):
    plt.scatter(PCA[inter[c], 0], PCA[inter[c], 1], s=2, color=color[c])
plt.title("Poly Kernel PCA (Uncentered)")
plt.xlabel('Component 1')
plt.ylabel('Component 2')
plt.axis('equal')
plt.show()
```



```
[29]: #RBF PCA (centered)
K_rbf,Kc_rbf = create_kernel_matrix(X, rbf_kernel,gamma=.01)
eigenvalues, eigenvectors=eig_sorted(Kc_rbf)
pvalues=np.real(eigenvalues[0:2])
pvectors=np.real(eigenvectors[:,0:2])
D=np.diag(1/np.sqrt(pvalues))
PCA=Kc_rbf@pvectors@D
inter=[slice(0,int(radius[0]*factor)),slice(int(radius[0]*factor),radius[1]*\
        factor),slice(radius[1]*factor,PCA.shape[0])]
for c in range(categories):
    plt.scatter(PCA[inter[c],0],PCA[inter[c],1],s=2,color=color[c])
plt.title("RBF Kernel PCA (Centered)")
plt.xlabel('Component 1')
plt.ylabel('Component 2')
plt.xlim(-1,1)
plt.ylim(-1,1)
plt.axis('equal')
plt.show()
```



```
[30]: #RBF PCA (uncentered)
K_rbf,Kc_rbf = create_kernel_matrix(X, rbf_kernel,gamma=.01)
eigenvalues, eigenvectors=eig_sorted(K_rbf)
pvalues=eigenvalues[0:2]
pvectors=eigenvectors[:,0:2]
D=np.diag(1/np.sqrt(pvalues))
PCA=K_rbf@pvectors@D
inter=[slice(0,int(radius[0]*factor)),slice(int(radius[0]*factor),radius[1]*\
        factor),slice(radius[1]*factor,PCA.shape[0])]
for c in range(categories):
    plt.scatter(PCA[inter[c],0],PCA[inter[c],1],s=2,color=color[c])
plt.title("RBF Kernel PCA (Uncentered)")
plt.xlabel('Component 1')
plt.ylabel('Component 2')
plt.xlim(-1,1)
plt.ylim(-1,1)
plt.axis('equal')
plt.show()
```



Bonus

1. Yes, the PCA and KPCA return the same results if KPCA uses a linear kernel. See plots below.
2. Let V be of maximal dimension d and compare the complexity of each algorithm. The PCA is

an optimization problem with cost function

$$\sum_{j=1}^d v_j \left(\frac{1}{n} \sum_{i=1}^n x_i x_i^\top \right) v_j, \text{ whereas the optimization problem for KPCA uses}$$
$$\frac{1}{n} \sum_{j=1}^d \alpha_j^\top K^2 \alpha_j.$$

Upon comparison of the two cost functions the difference in complexity is mainly given by the difference between calculating $\sum_{i=1}^n x_i x_i^\top$ vs. calculating K^2 . Let's analyse the number of operations for each. x_i has d rows so $x_i x_i^\top$ requires $d \times d$ products and the sum over n of them requires $n \times d^2$ products. For each entry in K^2 there are n products and n summations. Since there are n^2 matrix entries there are n^3 products and n^3 summations.

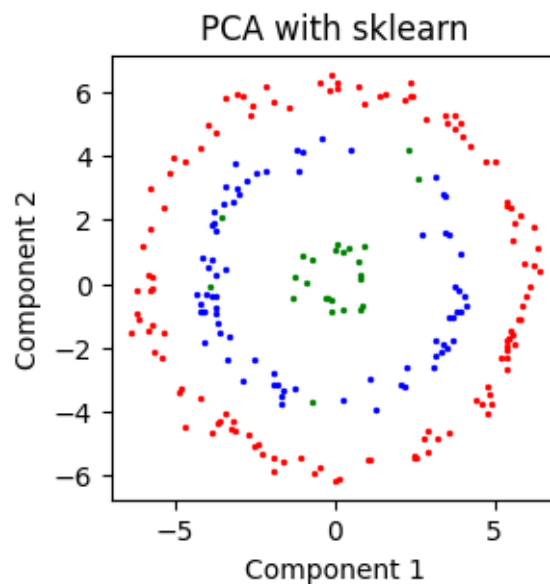
Since products are much more computationally expensive the difference between the two algorithms is between the calculation of $n \times d^2$ and n^3 products. Since d is generally less than n , PCA will generally be faster than KPCA for a linear kernel. Only when the dimension of V is large compared with the number of samples would KPCA be, in this case, a better choice.

Using the data problem 3 with 275 samples with $d=2$, the elapsed time for PCA was .78 ms and for KPCA .81 ms.

STAT672_Homework1_Bonus

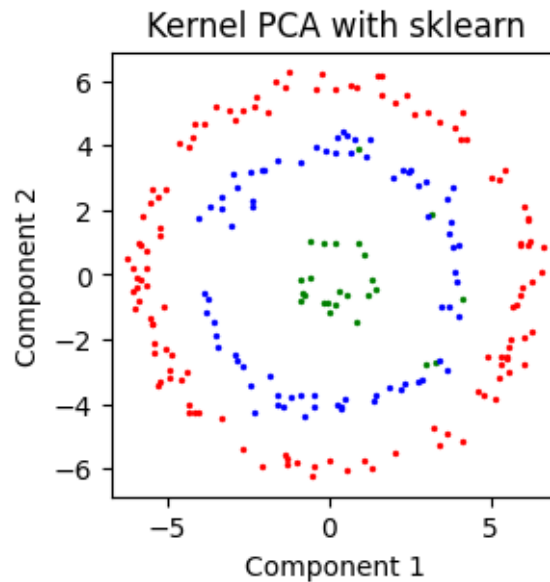
January 27, 2025

```
[ ]: #Construct X Matrix from data
X=np.hstack((Xp,Yp))
n_samples = X.shape[0]
start=time.time()
pca=PCA(n_components=2)
PCA=pca.fit_transform(X)
end=time.time()
elapsed_time=(end-start)
inter=[slice(0,int(radius[0]*factor)),slice(int(radius[0]*factor),radius[1]*\
        factor),slice(radius[1]*factor,n_samples)]
for c in range(categories):
    plt.scatter(PCA[inter[c],0],PCA[inter[c],1],s=2,color=color[c])
plt.title("PCA with sklearn")
plt.rcParams['figure.figsize'] = [3, 3]
plt.axis('equal')
plt.xlabel('Component 1')
plt.ylabel('Component 2')
plt.show()
print(f"elapsed time={elapsed_time} seconds")
```



elapsed time=0.0007951259613037109 seconds

```
[ ]: #Construct X Matrix from data
X=np.hstack((Xp,Yp))
n_samples = X.shape[0]
start=time.time()
kpca=KernelPCA(n_components=2, kernel='linear')
KPCA=pca.fit_transform(X)
end=time.time()
elapsed_time=(end-start)
inter=[slice(0,int(radius[0]*factor)),slice(int(radius[0]*factor),radius[1]*\
        factor),slice(radius[1]*factor,n_samples)]
for c in range(categories):
    plt.scatter(KPCA[inter[c],0],KPCA[inter[c],1],s=2,color=color[c])
plt.title("Kernel PCA with sklearn")
plt.rcParams['figure.figsize'] = [3,3]
plt.axis('equal')
plt.xlabel('Component 1')
plt.ylabel('Component 2')
plt.show()
print(f"elapsed time={elapsed_time} seconds")
```



elapsed time=0.0008051395416259766 seconds